

408 计算机组成原理学习笔记

当前进度：已完成《计算机组成原理-第1-4章-概述至指令系统.pdf》第 1 章全部可见正文内容整理。本轮阅读并处理 PDF 第 6-9 页，内容覆盖 1.2.4 后续“微体系结构”、1.2.5 计算机系统的不同用户、1.2.6 计算机系统的工作原理、1.3 计算机的性能指标，以及本章典型问题辨析。PDF 第 10 页已进入第 2 章，因此本 act 的第 1 章整理任务已完成。

下次任务：本章已完成，无需继续追加第 1 章内容；后续可按新的 act 处理下一章或其他科目。

第一章 计算机系统概述

本章是计算机组成原理的入门章，重点不是记一堆名词，而是先建立一条主线：**计算机系统由硬件和软件共同组成，硬件提供可执行能力，软件把人的问题逐层翻译成机器可执行的操作**。后续章节中的数据表示、存储器、指令系统、CPU、总线与 I/O，都会回到这条主线上。

本章需要优先掌握两类内容：

- 1. **计算机系统层次结构**：从应用问题到机器执行之间经历了哪些抽象层，每一层解决什么问题。
- 2. **计算机性能指标**：吞吐量、响应时间、CPU 时钟周期、主频、CPI、CPU 执行时间、MIPS、FLOPS 等指标之间如何区分和计算。

前半部分主要是“建立框架”，很多概念后面还会反复出现。初学时不用强行一次性吃透，但必须先知道它们在系统中的位置。

1.1 计算机发展历程

1.1.1 计算机硬件的发展

计算机硬件的发展可以从两条线理解：一条是**元器件更新换代**，另一条是**性能、容量和体系结构能力不断增强**。前者解释“为什么计算机越来越小、越来越快、越来越便宜”，后者解释“为什么计算机能支撑更复杂的软件和应用”。

1. 计算机的四代变化

代次	时间范围	主要元器件	典型特点
第一代	1946-1957 年	电子管	使用机器语言编程；主存多用延迟线或磁鼓；体积大、成本高、速度低
第二代	1958-1964 年	晶体管	运算速度明显提升；主存开始使用磁芯存储器；软件开始发展，出现高级语言和编译程序，并形成操作系统雏形
第三代	1965-1971 年	中小规模集成电路	半导体存储器逐步取代磁芯存储器；高级语言普及；操作系统进一步成熟，出现分时操作系统
第四代	1972 年至今	大规模、超大规模集成电路	微处理器出现；并行处理、流水线、高速缓存、虚拟存储器等关键技术广泛应用

这张表不建议死记年份，考试更常考的是“**某一代使用什么元器件、带来了什么系统能力变化**”。例如，晶体管替代电子管，不只是器件变小，还使可靠性、功耗、速度、成本都发生变化；集成电路的发展又为微处理器和个人计算机普及奠定了基础。

2. 计算机元件的更新换代

计算机硬件性能提升的核心原因，是晶体管、存储器和处理器技术持续演进。

1. 摩尔定律

在价格不变的前提下，集成电路上可容纳的晶体管数量大约每 18 个月翻一番，从而推动处理器性能和集成度快速提升。这里的重点不是“精确预测”，而是理解它揭示了信息技术快速发展的节奏。

2. 半导体存储器的发展

存储芯片容量从 KB 级逐步发展到 MB、GB、TB 级，容量提升使程序和数据能更大规模地放入存储系统中，支撑更复杂的软件和更大的数据处理任务。

3. 微处理器的发展

从 Intel 4004 开始，微处理器不断演进，出现了 8 位、16 位、32 位、64 位处理器。这里的“位数”通常指机器字长或通用寄存器宽度，它影响一次整数运算可处理的数据位数，也会影响可直接寻址的内存空间大小。

初学时可以把“字长”理解为 CPU 一次“自然处理”的二进制数据宽度。字长越大，并不必然意味着所有程序都越快，但它会影响整数运算能力、地址空间和指令/数据处理方式。

1.1.2 计算机软件的发展

软件的发展与硬件的发展相互促进。硬件能力增强后，软件可以承担更复杂的任务；软件需求提升，又会反过来推动硬件体系结构改进。

计算机语言大致经历了如下演进：

层次	特点	典型代表
机器语言	由二进制指令组成，计算机能直接识别和执行，但人类编写困难	二进制指令序列
汇编语言	用助记符表示机器指令，可读性提高，但仍强依赖具体机器	mov、add 等助记符
高级语言	更接近人的表达方式，提高开发效率，需要翻译成机器可执行形式	FORTRAN、Pascal、C++、Java 等

系统软件也随之发展。操作系统是其中最重要的一类，它负责管理硬件资源，并向应用程序提供更方便的运行环境。常见系统软件还包括数据库管理系统、语言处理程序、网络与分布式软件系统、标准库程序和服务性程序等。

本节要抓住一个关键点：**软件并不是脱离硬件单独存在的，软件最终必须通过硬件执行；硬件也不是直接面向用户问题的，硬件需要软件把复杂任务组织起来。**

1.2 计算机系统层次结构

1.2.1 计算机系统的组成

一个完整的计算机系统由**硬件**和**软件**共同组成。

- **硬件**：有形的物理装置，包括 CPU、主存储器、外部设备、总线等。
- **软件**：在硬件上运行的程序，以及与程序相关的数据和文档。

计算机系统的实际性能，往往取决于软件对硬件资源的利用效率，而这种效率又受硬件能力限制。因此设计计算机系统时，需要合理划分软硬件功能边界。

一个重要原则是：

- 对于**使用频繁、硬件实现成本较低、对性能影响明显**的功能，适合交给硬件实现。
- 对于**变化较多、逻辑复杂、灵活性要求高**的功能，适合交给软件实现。

这也是后续学习中经常遇到的思想：同一个功能既可能由硬件完成，也可能由软件模拟完成，关键看性能、成本、复杂度和可维护性之间的权衡。

1.2.2 计算机硬件

硬件部分可以从两个层次理解：先看冯·诺依曼计算机的基本思想，再看现代计算机硬件系统的功能部件。

1. 冯·诺依曼机的基本思想

考点追踪：冯·诺依曼机的特点（2019）

冯·诺依曼在研究 EDVAC 机时提出了“存储程序”的思想，这一思想奠定了现代计算机的基本结构。基于该思想的计算机通常称为冯·诺依曼机。

它的核心特点可以概括为：

1. 采用**存储程序**工作方式：程序和初始数据预先存入主存储器，机器启动后自动、连续地取指并执行，直到程序结束。
2. 硬件系统由五大部件组成：**运算器、控制器、存储器、输入设备、输出设备**。
3. 指令和数据都存放在存储器中，形式上都用二进制编码表示，只有在执行时才根据内容和使用场景区分“这是指令”还是“这是数据”。
4. 指令和数据均采用二进制编码。
5. 指令由**操作码和地址码**组成：操作码说明“做什么”，地址码说明“对谁做”或“操作数在哪里”。

这部分是本章最重要的概念之一。尤其要理解“存储程序”不是简单地把程序放进内存，而是意味着计算机可以按照内存中的指令序列自动执行，从而摆脱人工逐步干预。

2. 计算机的功能部件

现代计算机通常将运算器、控制器和各类寄存器高度集成到 CPU 芯片中。完整的硬件系统主要包括：CPU、存储器、输入/输出控制器、外部设备，以及连接这些部件的总线。

功能部件	作用	学习重点
CPU	负责取指、译码、执行指令，是执行控制和运算的核心	运算器、控制器、寄存器、数据通路
存储器	存放程序和数据	主存、Cache、外存的层次关系
外部设备与设备控制器	完成输入/输出，并通过控制器与主机通信	I/O 设备、I/O 控制器、接口
总线	在各部件之间传送信息	数据总线、地址总线、控制总线的配合

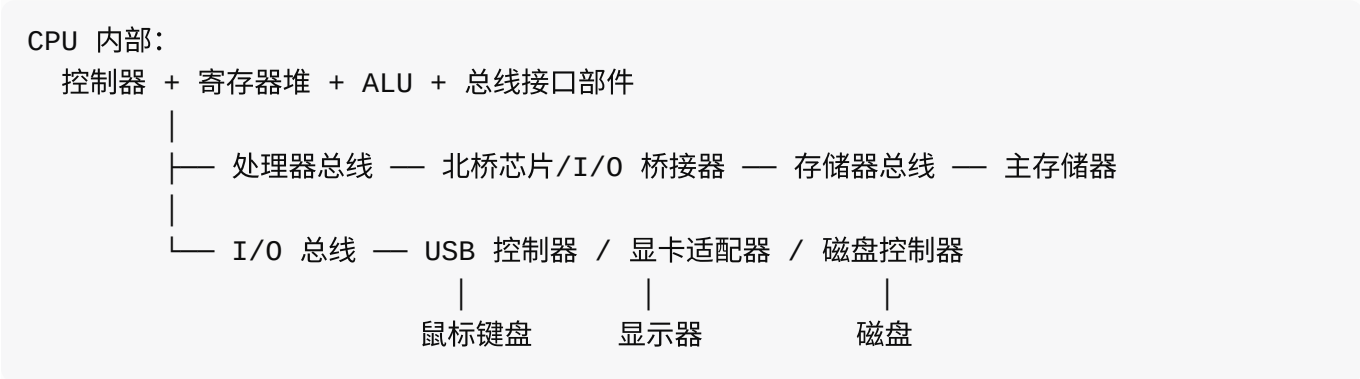
CPU 内部可以进一步理解为两大部分：

- 1. **数据通路**：真正完成数据传送和运算的硬件通路，核心包括 ALU 和通用寄存器组。ALU 负责算术与逻辑运算，寄存器组暂存操作数和中间结果。
- 2. **控制单元**：从存储器取出指令并译码，根据指令语义产生控制信号，指挥数据通路按正确顺序完成操作。

存储器也要区分层次。现代内存系统通常包含主存和高速缓存 Cache，但在传统说法中，“内存”往往专指主存。外存包括磁盘、固态硬盘等可与主存交换数据的设备，也包括磁带、光盘等适合长期备份的存储介质。

外部设备通过设备控制器连接到主机。例如键盘接口、显示控制器、网络控制器等都属于设备控制器。它们的作用是屏蔽具体设备的物理差异，让 CPU 能以统一方式进行 I/O 控制。

图 1.1 的多总线硬件系统可以简化理解为：



这个图表达的重点不是记住某一种具体连接方式，而是理解：CPU、主存和 I/O 设备之间不是随意相连的，而是通过总线、桥接器、接口和控制器形成互连结构，完成全系统的数据传输与协调。

1.2.3 计算机软件

1. 系统软件和应用软件

软件按功能可以分为系统软件和应用软件。

类型	含义	典型例子
系统软件	保障计算机系统高效、正确运行的基础软件，负责管理和调度系统资源	操作系统、数据库管理系统、语言处理程序、标准库、网络与分布式软件系统
应用软件	面向具体应用领域，为用户解决特定问题的软件	科学计算软件、工程设计软件、数据统计与信息处理软件

系统软件可以理解为“让计算机好用、可管理、可运行程序”的基础层；应用软件则是“直接解决用户问题”的功能层。

2. 软件和硬件的逻辑功能等价性

在计算机中，某些功能既可以由硬件实现，也可以由软件实现。从逻辑功能看，只要输入输出结果一致，二者可以等价；但从效率看，硬件实现通常更快，软件实现通常更灵活。

例如浮点运算可以由专用浮点硬件完成，也可以由软件子程序模拟完成。在符合相同数值规范的前提下，二者结果可以一致，但硬件实现效率通常更高。

因此，软硬件逻辑功能等价性是计算机系统设计的重要依据。系统设计时要考虑：

- 哪些功能应由硬件承担，以提高性能；
- 哪些功能应由软件实现，以提高灵活性和可维护性；
- 设计目标、成本收益、技术可行性之间如何平衡。

1.2.4 计算机系统的层次结构

计算机系统采用多级层次结构，本质上是在复杂硬件和用户应用之间建立一系列抽象层。每一层都向上一层提供较简洁的接口，向下一层依赖更底层的功能实现。

图 1.2 可以整理为如下层次：

自上而下的层次	主要面向对象	大致职责
应用（问题）	最终用户	描述和解决用户关心的实际问题
算法与编程	程序员	把问题抽象成算法，并用编程语言表达
操作系统/虚拟机	程序员、系统软件	管理硬件资源，提供更易用的运行环境
指令集体系结构（ISA）	架构师、编译器/汇编程序	定义软件能直接使用的硬件功能接口
微体系结构	架构师	决定 ISA 如何在处理器内部实现
功能部件/RTL	硬件设计人员	用寄存器传送级部件实现微结构功能
电路与器件	电子工程师	用门电路、触发器、晶体管等实现硬件基础

从学习角度看，这张层次图非常重要。用户提出的是应用问题，程序员写的是高级语言程序，机器最终执行的是指令和底层电路动作。中间的每一层都在做“翻译”和“抽象”：向上隐藏复杂实现，向下依赖底层机制。

1. 算法和编程

解决应用问题时，首先要把问题抽象成算法。算法描述“如何一步步解决问题”，编程语言则把算法表达成程序。

编程语言可以分为高级语言和低级语言：

考点追踪：三种编程语言的特点（2015、2024）

语言类型	特点	是否能被硬件直接执行
机器语言	由 0 和 1 组成的指令序列，机器能直接识别，程序员编写和记忆困难	能
汇编语言	用英文助记符或缩写表示机器指令，可读性比机器语言更好	不能，需要汇编程序翻译
高级语言	接近自然语言或数学表达，便于描述问题和提高开发效率	不能，需要编译或解释成机器可执行形式

注意不要把“汇编语言更接近机器”误解为“汇编语言能被硬件直接执行”。硬件直接执行的是机器语言，汇编语言必须先翻译成机器语言。

2. 翻译程序

考点追踪：各种翻译程序的概念（2016）

高级语言程序不能被 CPU 直接执行，必须经过语言处理系统转换为机器语言程序。常见翻译程序包括汇编程序、解释程序和编译程序。

翻译程序	输入	输出或执行方式	核心区别
汇编程序	汇编语言源程序	机器语言目标程序	面向汇编语言到机器语言的转换
解释程序	高级语言源程序	逐条翻译并立即执行，不生成独立目标程序	边翻译边执行
编译程序	高级语言源程序	一次性翻译成汇编语言或机器语言目标程序	先生成目标程序，再运行目标程序

三者最容易混淆的是解释程序和编译程序。记忆方法是：**解释程序强调“逐条翻译、立即执行”，编译程序强调“一次翻译、生成目标程序”。**

3. 操作系统

操作系统为程序运行提供环境。语言处理系统本身也必须在操作系统提供的运行环境中执行。
从层次结构看，操作系统通过对硬件及底层结构进行抽象，构建出一台更方便程序员使用的“虚拟机”。这台虚拟机隐藏了许多硬件细节，例如设备管理、文件管理、进程管理和内存管理，使应用程序不必直接面对复杂硬件。

4. 指令集体系结构

指令集体系结构（Instruction Set Architecture, ISA）是计算机软件和硬件之间的关键接口。它从程序员和编译器的视角，定义了软件可以直接使用的硬件功能。

ISA 主要规定：

- 指令格式；
- 操作类型；
- 寻址方式；
- 可访问的寄存器等硬件资源。

可以把 ISA 理解为“软件能看到的计算机功能视图”。我们写的机器语言程序，本质上就是一串符合 ISA 规定的指令序列；硬件执行程序的过程，就是逐条解释并完成这些指令所规定操作的过程。

5. 微体系结构

微体系结构又称微架构，它回答的问题是：**在已经确定 ISA 的前提下，处理器内部到底怎样组织硬件，才能实现这些指令功能。**

可以用一句话区分 ISA 和微体系结构：

- **ISA 规定“做什么”**：有哪些指令、指令格式怎样、寻址方式怎样、程序员能看到哪些寄存器等。

- **微体系结构决定“怎么做”**：数据通路怎样组织、控制单元怎样产生控制信号、流水线分几级、Cache 层次怎样设计、分支预测机制怎样实现等。

例如，加法指令在 ISA 层面只说明“完成两个操作数相加并写回结果”；而在微体系结构层面，要决定它通过串行进位加法器、超前进位加法器，还是专用 SIMD 单元实现。不同实现方式可能性能、面积、功耗差异很大，但只要它们遵循同一套 ISA 规范，软件通常就能在这些处理器上运行。

这也是为什么同属于 Intel x86 系列的处理器可以运行相同机器程序，却在性能、功耗、流水线结构、缓存结构等方面差异明显。对考研学习来说，后续 CPU 章节会重点展开“数据通路、控制器、流水线”等内容，本节先建立位置感即可。

层次	关注问题	典型内容
ISA	软件可见的硬件接口	指令格式、寄存器、寻址方式、数据类型
微体系结构	ISA 的硬件实现方式	数据通路、控制器、流水线、Cache、分支预测
功能部件/电路	具体硬件模块和电路实现	ALU、寄存器、门电路、触发器、晶体管

初学时最容易犯的错误，是把“体系结构”和“组成实现”混为一谈。体系结构偏向程序员可见的抽象规范，微体系结构和组成实现偏向硬件内部细节。两台机器可以有相同 ISA，但内部微体系结构不同；也可以因为 ISA 不同，而导致软件二进制不兼容。

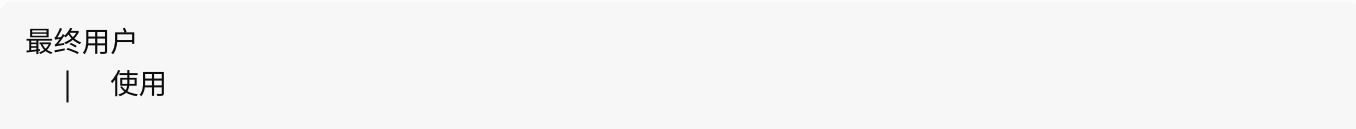
1.2.5 计算机系统的不同用户

计算机系统采用层次结构后，不同用户看到的是不同抽象层。这里的“用户”不是只指最终使用电脑的人，而是指在不同层次上与计算机系统打交道的人。

用户类型	主要工作	关注层次	是否需要理解底层细节
最终用户	使用应用程序完成任务，如办公、浏览网页、购物	应用程序层、操作系统界面	通常不需要
系统管理员	安装、配置、管理和维护系统	操作系统层、硬件资源管理	需要理解系统运行和资源管理
应用程序员	使用高级语言开发应用软件	高级语言、库、操作系统接口	需要理解语言和系统接口，通常不需要硬件细节
系统程序员	开发操作系统、编译器、数据库等核心系统软件	操作系统、编译器、ISA	需要理解软硬件交界处
硬件设计人员	设计处理器、存储器、I/O 接口和电路	ISA 以下、微体系结构和电路	需要深入理解硬件实现

不同用户处在不同层次，看到的“机器”也不同。最终用户看到的是应用程序和操作系统界面；应用程序员看到的是高级语言和系统调用；系统程序员会更接近 ISA；硬件设计人员则关心 ISA 以下的具体实现。

可以把图 1.3 的层次关系整理为：



应用程序
| 依赖
高级语言程序 / 应用开发环境
| 编译、解释、运行时支持
操作系统、编译器、汇编器
| 面向
指令集体系结构 ISA
| 由硬件实现
CPU、主存、I/O 设备
| 由更底层实现
数字逻辑与电路设计

本节最重要的概念是**透明性**：下层机器的某些结构特性对上层用户不可见，上层用户就可以在不知道底层细节的情况下完成工作。例如，应用程序员写 C 程序时通常不需要知道 CPU 内部的控制信号如何产生，也不需要知道主存芯片内部如何读写；这些细节对他来说是“透明的”。

不过，“透明”不等于“不存在”，而是“当前层次的用户不必直接感知”。后续学习中很多机制都具有这种特点，例如 Cache 对多数程序员透明，虚拟存储器对应用程序提供更大的虚拟地址空间，I/O 系统把复杂设备抽象成统一接口。

1.2.6 计算机系统的工作原理

1. “存储程序”工作方式

考点追踪：存储程序工作方式、程序执行过程相关理解

存储程序工作方式的核心是：**程序执行前，指令和数据都预先装入主存；程序开始后，计算机无需人工逐条干预，而是自动地逐条取出并执行指令。**

一条指令的执行通常可以概括为四个基本阶段：

根据 PC 取指令
↓
指令译码，同时更新 PC
↓
取操作数并执行
↓
写回或送出结果
↓
继续根据新的 PC 取下一条指令

其中，PC 是程序计数器，用来给出下一条将要执行的指令地址。程序开始执行前，第一条指令的地址会被送入 PC；取指时，CPU 根据 PC 的内容访问主存，取出指令。

每条指令执行结束后，PC 的更新方式取决于指令类型：

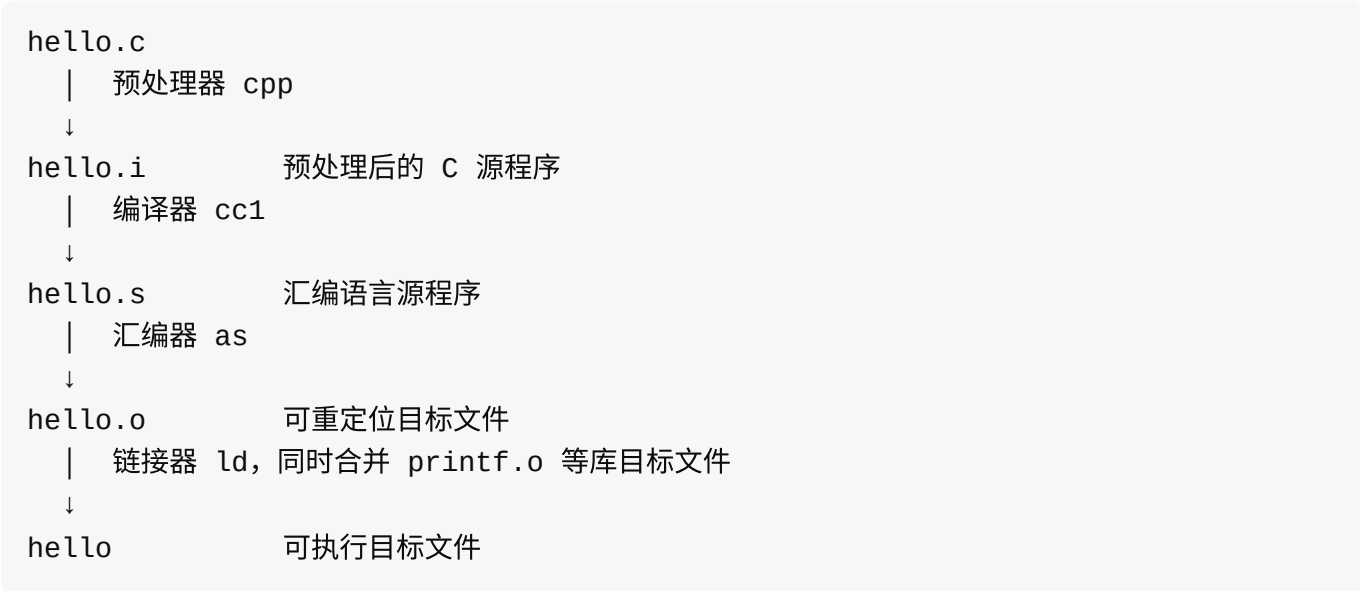
- 如果是**顺序执行指令**，下一条指令地址通常等于当前 PC 加上指令长度。
- 如果是**跳转或分支指令**，下一条指令地址由指令中指定的目标地址或分支条件决定。

这里要特别注意：程序并不是“CPU 随便往下读内存”，而是由 PC 控制指令流。顺序结构、循环结构、条件分支，本质上都要落实到 PC 的改变上。

2. 从源程序到可执行文件

考点追踪：翻译过程的四个阶段（2022）

以 C 语言程序 `hello.c` 为例，源程序不能直接被 CPU 执行。它必须经过预处理、编译、汇编和链接，最终形成可执行目标文件。



四个阶段的作用如下：

阶段	输入	输出	主要作用
预处理	hello.c	hello.i	处理以 # 开头的预处理指令，如展开头文件、宏替换等
编译	hello.i	hello.s	将高级语言语句翻译成汇编语言语句
汇编	hello.s	hello.o	将汇编语言转换为机器语言，生成可重定位目标文件
链接	hello.o 与库目标文件	hello	合并多个目标文件和库函数，生成可执行目标文件

这条链条很适合用来理解“软件最终如何变成硬件可执行的东西”。程序员写的是高级语言文本，CPU 执行的是二进制机器指令；中间每一步都在降低抽象层次，直到生成机器可以装入内存并执行的文件。

考试中常见混淆点如下：

- 1. **预处理不是编译**：预处理主要处理宏、头文件、条件编译等文本层面的内容。
- 2. **编译结果不一定直接是机器码**：这里的编译阶段生成的是汇编程序 `hello.s`。
- 3. **汇编器输出的是目标文件，不一定已经能独立运行**：`hello.o` 仍可能需要链接外部库函数。
- 4. **链接解决“组合问题”**：多个目标文件、库函数、外部符号等要通过链接整合成最终可执行文件。

1.3 计算机的性能指标

1.3.1 主要性能指标与计算公式

计算机性能不能只用“快”来描述。不同场景关注点不同：用户可能关心一个程序多久能完成，服务器可能关心单位时间能处理多少请求，科学计算可能关心每秒能完成多少浮点运算。因此，本节要把几个指标区分清楚。

1. 吞吐量和响应时间

指标	含义	适合描述的场景
吞吐量	单位时间内系统完成任务的数量	服务器处理请求、批处理任务、I/O 系统处理数据
响应时间	从任务提交到结果返回所经历的总时间	用户运行程序、打开网页、执行一次命令

吞吐量和响应时间都能反映性能，但角度不同。提高吞吐量不一定等价于缩短单个任务响应时间。例如，多核服务器可以同时处理更多任务，使吞吐量上升；但如果某个单独程序没有并行化，它的响应时间未必明显缩短。

2. CPU 时钟周期、主频与 CPI

CPU 内部由时钟信号协调工作。理解性能公式前，必须先区分三个量：

指标	含义	关系
时钟周期	一个时钟脉冲所对应的时间长度	主频越高，时钟周期越短
主频	CPU 每秒产生多少个时钟周期	主频 = 1 / 时钟周期
CPI	平均每条指令需要的时钟周期数	与指令类型、处理器结构、存储访问等有关

如果一条指令平均需要多个时钟周期才能完成，就不能只看主频来判断性能。主频高但 CPI 也高，最终运行时间不一定短；主频略低但 CPI 更小，也可能执行更快。

3. CPU 执行时间

考点追踪：CPU 执行时间的相关计算（2012、2013、2014、2017、2022、2023）

运行一个程序所花费的 CPU 时间可以写成：

$$\begin{aligned} CPU \text{ 执行时间} &= \frac{CPU \text{ 时钟周期数}}{\text{主频}} \\ &= \frac{\text{指令条数} \times CPI}{\text{主频}} \end{aligned}$$

这条公式是本章计算题的核心。它告诉我们，CPU 性能主要受三个因素共同影响：

- 1. **指令条数**：程序被翻译后需要执行多少条机器指令。
- 2. **CPI**：平均每条指令需要多少个时钟周期。
- 3. **主频**：每秒能提供多少个时钟周期。

三者之间存在制约关系。例如，复杂指令集可能减少指令条数，但使控制逻辑复杂、CPI 增大或主频难以提高；精简指令集可能增加指令条数，但便于流水线设计、降低 CPI 或提高主频。考题经常

借此考查“不能只看某一个指标”。

典型分析：主频提升不等于性能同比例提升

已知机器 M1 和 M2 具有相同指令集，M1 主频为 2GHz，程序 P 在 M1 上运行 10s。M2 采用新技术可提高主频，但平均 CPI 增加到 M1 的 1.5 倍。若希望 P 在 M2 上运行时间缩短为 6s，求 M2 主频至少是多少。

分析如下：

$$\begin{aligned} M1 \text{ 的时钟周期数} &= CPU \text{ 执行时间} \times \text{主频} \\ &= 10s \times 2GHz \\ &= 2 \times 10^{10} \end{aligned}$$

因为两机 ISA 相同，程序的指令条数相同；M2 的平均 CPI 为 M1 的 1.5 倍，所以 M2 所需时钟周期数为：

$$1.5 \times 2 \times 10^{10} = 3 \times 10^{10}$$

若运行时间要求为 6s，则：

$$\text{主频} = \frac{3 \times 10^{10}}{6s} = 5GHz$$

所以 M2 主频至少为 5GHz。它的主频是 M1 的 2.5 倍，但实际执行时间只是从 10s 缩短到 6s，性能提升为：

$$\frac{10}{6} \approx 1.67$$

这道题的关键不是代公式，而是理解：**CPI 增大抵消了一部分主频提升带来的收益。**

4. IPS、MIPS 与 FLOPS

考点追踪：MIPS 相关计算（2012、2013）

考点追踪：浮点运算指标的概念（2011、2021）

IPS 表示每秒执行多少条指令：

$$IPS = \frac{\text{主频}}{\text{平均 } CPI}$$

MIPS 表示每秒执行多少百万条指令：

$$MIPS = \frac{\text{指令条数}}{CPU \text{ 执行时间} \times 10^6} = \frac{\text{主频}}{CPI \times 10^6}$$

MIPS 看起来直观，但用于不同机器之间比较时有明显缺陷：不同机器的 ISA 可能不同，同一高级语言语句在不同机器上可能被翻译成不同数量、不同复杂度的机器指令；即使 MIPS 高，也不一定说明实际程序运行更快。

FLOPS 表示每秒执行的浮点运算次数，常用于科学计算、图形处理和人工智能训练等需要大量浮点运算的场景。常见单位如下：

单位	含义
MFLOPS	10^6 次浮点运算/秒
GFLOPS	10^9 次浮点运算/秒
TFLOPS	10^{12} 次浮点运算/秒
PFLOPS	10^{15} 次浮点运算/秒
EFLOPS	10^{18} 次浮点运算/秒
ZFLOPS	10^{21} 次浮点运算/秒

需要注意单位前缀在不同语境中的差异：在存储容量、文件大小中，K、M、G、T 常按 2 的幂次理解，如 $1\text{KB} = 2^{10}\text{B}$ ；在描述速率、频率和 FLOPS 时，K、M、G、T 通常按 10 的幂次理解，如 $1\text{GHz} = 10^9\text{Hz}$ 。

1.3.2 基准程序

基准程序是专门用于性能评测的一组典型程序，用来模拟真实应用负载，从而比较不同计算机系统在实际场景中的运行效率。

使用基准程序时要抓住两点：

1. **它比单纯比较主频、MIPS 更接近真实应用。**因为它运行的是完整程序，能同时反映 CPU、存储器、编译器、操作系统等因素的综合影响。
2. **它仍然不是绝对标准。**一个基准程序只能代表某类工作负载，如果工作负载变化，性能结论可能改变。

因此，看到“某机器基准程序执行更快”时，比较稳妥的表述是：它在该基准程序代表的负载下性能更好，但不能直接推出它在所有场景下都更好。

1.4 本章典型问题辨析

1.4.1 编译、解释与汇编的区别

编译程序、解释程序和汇编程序都属于翻译程序，但它们的输入对象、输出结果和执行方式不同。

程序	输入	处理方式	结果
编译程序	高级语言源程序	一次性整体翻译	生成目标程序，目标程序可独立执行或进一步链接
解释程序	高级语言源程序	逐条翻译并立即执行	通常不生成可长期保存的目标程序
汇编程序	汇编语言源程序	将助记符形式的指令翻译为机器指令	生成机器语言目标程序

判断一个翻译程序属于哪一类，关键看**源语言是什么**。若源语言是 Java、C 等高级语言，目标语言是机器语言或汇编语言，称为编译程序；若源语言是汇编语言，目标语言是机器语言，称为汇编程序。

这里还要补充一句：实际系统中可能同时混合使用编译和解释思想，例如先把源程序编译成中间表示或字节码，再由虚拟机解释执行或即时编译执行。考试中按教材基本概念区分即可。

1.4.2 “透明性”的含义

“透明性”在计算机领域不是“看得见”，而是指：**某种事物或属性虽然存在，但从某类用户的视角看不到、也不必关心。**

例如：

- 对高级语言程序员来说，机器指令格式、具体机器结构、底层数据表示细节通常是透明的。
- 对汇编语言程序员或机器语言程序员来说，CPU 内部的指令寄存器 IR、存储器地址寄存器 MAR、存储器数据寄存器 MDR 等通常也是透明的。
- 对最终用户来说，编译、链接、系统调用、内存管理等底层机制通常是透明的。

透明性体现了层次结构的价值：上层只需要使用下层提供的接口，不必了解所有实现细节。它能降低使用难度，但并不代表底层机制不重要。学习计算机组成原理，就是在逐步揭开这些“透明”的底层实现。

1.4.3 计算机体系结构与计算机组成的区别和联系

计算机体系结构和计算机组成关系密切，但侧重点不同。

对比项	计算机体系结构	计算机组成
关注对象	程序员可见的机器属性	体系结构的具体硬件实现
典型内容	指令集、数据类型、寻址方式、寄存器可见性	数据通路、控制器、乘法器实现、总线连接、流水线组织
所处层次	抽象规范层	具体实现层
对软件兼容性的影响	直接影响二进制程序能否运行	通常不改变软件可见接口，但影响性能和成本

例如，“是否提供乘法指令”属于体系结构问题，因为程序员和编译器能看到这条指令；“乘法指令用阵列乘法器还是移位相加电路实现”属于组成问题，因为它是硬件内部如何完成该指令的实现细节。

因此，两台机器可以有相同体系结构，但组成方式不同；它们能运行相同机器程序，却可能在性能、功耗和成本上差异明显。

1.4.4 如何正确看待基准程序性能

基准程序执行得快，通常说明机器在该基准程序代表的负载下性能较好，但不能绝对说明机器在所有任务中都更好。

原因是不同程序对资源的需求不同：

- 计算密集型程序更依赖 CPU 运算能力和流水线效率；
- 存储访问密集型程序更依赖 Cache、主存带宽和访存延迟；
- I/O 密集型程序更依赖磁盘、网络、设备控制器和操作系统调度；
- 浮点密集型程序更适合用 FLOPS 相关指标衡量。

所以，性能评价不能只看单一指标，也不能只看单一程序。比较可靠的思路是：**先明确工作负载，再选择合适指标和基准程序，最后结合响应时间、吞吐量、CPU 执行时间等多方面综合判断。**

当前进度：已完成计算机组成原理 PDF 第 6 章全部可见正文内容整理。本轮阅读和处理 PDF 第 31-32 页，对应教材页第 284-285 页，补充完成 6.2 总线事务和定时，重点包括非突发传输、突发传输、串行/并行传输、同步定时、异步定时、半同步定时和分离式定时。PDF 第 33 页已进入第七章，因此第六章笔记已完成。

下次任务：本章已完成，无需继续推进；后续可进入计算机组成原理 PDF 第 7 章笔记整理。

第六章 总线

第六章围绕“总线”展开。总线是连接计算机各部件的公共信息通道，重点通常不在复杂推导，而在**概念辨析、性能指标计算、传输方式和定时方式比较**。选择题很容易把“数据总线传什么”“地址线决定什么”“串行一定慢不慢”“并行一定快不快”等问题混在一起考。

本章复习时可以抓住两条主线：

1. **为什么需要总线**：为了减少部件之间直接连线的数量，让多个部件能通过一组公共线路交换信息。
2. **总线带来的矛盾**：共享总线会降低连线复杂度，但会带来竞争、仲裁、带宽瓶颈、时序协调等问题。

6.1 总线概述

早期计算机中，各部件之间通过专用连线直接互连，这种方式也称为**分散连接**。随着 I/O 设备种类和数量增多，分散连接方式会让连线数量急剧增加，系统扩展、维护和标准化都变得困难。

因此，现代计算机逐步采用**总线连接方式**：用一组公共信息传输线路连接多个功能部件，各部件在不同时间共享这组线路完成数据、地址、控制信息的交换。

总线的核心特征可以概括为：

特征	含义	考试理解
分时性	同一时刻通常只允许一个部件向总线发送信息	多个发送方要通过仲裁或约定分时使用总线
共享性	多个部件都挂接在总线上，并可接收总线上的信息	降低连线数量，但也容易形成瓶颈

考点追踪：总线相关的概念与特点（2016、2017）

理解总线时，不要只把它想成“一根线”。更准确地说，总线是一组线路及其协议的组合，包括传输哪些信息、一次传多少位、谁先使用、何时发送、何时接收等规则。

6.1.1 总线的分类

总线可以从不同角度分类。对考研来说，最重要的是按**位置**和**连接对象**分类，以及系统总线内部按**传输信息内容**分类。

1. 内部总线

内部总线也称片内总线，指芯片内部的总线，用于 CPU 芯片内部各寄存器之间，以及寄存器与 ALU 之间的数据传送。

内部总线的特点是距离短、速度快、受 CPU 内部结构约束明显。它更多服务于 CPU 内部的数据通路，不直接用于连接主存和外部设备。

2. 系统总线

系统总线用于连接计算机系统各功能部件，如 CPU、主存和 I/O 接口。按照所传输信息的内容不同，系统总线通常分为三类：

类型	主要作用	方向特点	容易混淆点
数据总线	在各部件之间传输数据信息	通常双向	“数据”不只包括操作数，也可能包括指令、状态字、中断类型号等
地址总线	指出要访问的主存单元或 I/O 端口地址	通常单向，由 CPU 发出	位数决定可寻址空间大小
控制总线	传输控制、状态、应答和时序协调信号	单根线方向可不同，整体可看作双向信号集合	不要简单说控制总线一定单向或一定双向

考点追踪：数据总线上传输的内容（2011）

数据总线的位数反映 CPU 一次能够并行传送的数据位数。例如，若数据总线宽度为 32 位，则一次并行传输可传送 32 bit 数据。这里的“数据”是广义数据，不限于算术运算中的操作数。

地址总线的位数决定系统最大可寻址空间。例如，若地址总线有 16 位，且每个地址对应 1B 存储单元，则最大可寻址空间为：

$$2^{16} = 65536B = 64KB$$

控制总线用于传输各种控制和协调信号，如时钟、复位、总线请求/允许、中断请求/响应、读/写命令、传输确认等。控制总线由多根控制信号线组成，不同信号线的方向可能不同，因此要按具体信号分析。

3. I/O 总线

I/O 总线用于连接主机与内部各类 I/O 控制器，例如显卡、网卡等。它通常采用标准化的内部总线协议，典型标准包括 PCI、AGP、PCI Express 等。

I/O 总线强调主机内部扩展能力，核心作用是让不同厂商的外设控制器能够按照统一规范接入系统。

4. 通信总线

通信总线用于主机与外部 I/O 设备之间，或不同计算机系统之间的通信。由于传输距离、传输速率、电气规范等要求不同，通信总线更强调接口标准和通信协议，典型代表包括 RS-232、USB 等。

本小节要特别注意两个层次：

- 1. 系统总线内部三分法：数据总线、地址总线、控制总线。
- 2. 从连接范围看总线：内部总线、系统总线、I/O 总线、通信总线。

6.1.2 常见的总线标准

总线标准是国际上制定的、用于互连计算机各功能模块的规范，本质上是一组接口协议。总线标准规定了连接方式、信号含义、时序、电气特性、传输宽度、传输速率等内容。

说明：PDF 脚注指出，本节“常见的总线标准”内容自 2021 年统考大纲中删除，仅供学习参考。复习时应以理解典型标准的分类和特征为主，不必死记大量英文全称。

考点追踪：总线标准的英文缩写（2010）

常见总线标准可以按“系统总线、局部总线、设备总线、串行/并行”等维度整理：

总线标准	英文含义或常用名称	主要特点	归类理解
ISA	Industry Standard Architecture	早期系统总线标准，速度低，CPU 占用率高，占用较多中断资源	系统总线、并行总线
EISA	Extended Industry Standard Architecture	ISA 的扩展，支持多主控器和突发传送，兼容 ISA	系统总线、并行总线
VESA	Video Electronics Standards Association	32 位局部总线标准，面向高速图像数据传输	局部总线、并行总线
PCI	Peripheral Component Interconnect	32 位或 64 位高性能外围总线，常用于显卡、声卡、网卡等扩展卡	局部总线、并行总线
AGP	Accelerated Graphics Port	面向图形卡的高速接口，允许显卡高效访问系统主存	局部总线、并行总线
PCI-E	PCI-Express	高速串行点对点连接，由多条通道组成，支持全双工通信	局部总线、串行总线
RS-232C	串行通信接口标准	用于 DTE 与 DCE 之间的串行二进制通信	设备总线、串行总线
USB	Universal Serial Bus	通用串行总线，支持即插即用、热插拔、多设备级联	设备总线、串行总线
PCMCIA	Personal Computer Memory Card International Association	主要用于笔记本电脑扩展卡	设备总线、并行总线
IDE/ATA	Integrated Drive Electronics / ATA	连接主板与硬盘、光驱的传统并行接口	设备总线、并行总线
SCSI	Small Computer System Interface	高性能系统级设备接口，常用于服务器硬盘等	设备总线、并行总线
SATA	Serial ATA	IDE/ATA 的串行替代标准，采用高速串行传输	设备总线、串行总线

考点追踪：区分设备总线和局部总线（2013）

局部总线强调**靠近 CPU 的高速扩展通路**，常用于显卡等高速部件；设备总线强调**外部或内部设备接口标准**，常用于硬盘、USB 设备等外设连接。

考点追踪：PCI-E 总线的特点（2017）

PCI-E 与传统 PCI 的关键区别在于：PCI 是并行总线，而 PCI-E 是高速串行点对点连接。PCI-E 可由多条通道组成，如 x1、x4、x8、x16，并支持全双工通信。考试中若出现“串行但高速”“点对点”“多通道”“全双工”，通常要联想到 PCI-E。

考点追踪：USB 总线的特点（2012）

USB 是通用串行总线，主要面向外部 I/O 设备，常见特点包括即插即用、热插拔、支持多设备连接、使用方便。不要把 USB 误判为并行总线。

6.1.3 总线的性能指标

总线性能指标主要用于衡量总线传输能力。重点不在死记概念，而在区分“时钟频率、工作频率、总线宽度、总线带宽”之间的关系。

1. 总线传输周期

总线传输周期是完成一次完整总线事务的总时间，例如一次读操作或一次写操作所需的时间。它通常由若干个总线时钟周期组成，因此也可简称为总线周期。

2. 总线时钟频率

总线时钟频率是总线基础时钟信号的频率，也就是总线时钟周期的倒数。早期总线的时钟可能与 CPU 时钟同步，但随着 CPU 速度提高，现代总线时钟通常独立于 CPU 时钟。

3. 总线工作频率

总线工作频率指总线每秒能够完成的有效数据传输次数。它不一定等于总线时钟频率。

例如，某些总线在一个时钟周期内可以传输 2 次、4 次甚至更多次数据，则总线工作频率可以达到时钟频率的 2 倍、4 倍等。

4. 总线宽度

总线宽度指总线中数据线的条数，决定每次能够并行传输的数据位数。例如，16 位总线一次可并行传输 16 bit，也就是 2B。

5. 总线带宽

总线带宽指总线在单位时间内所能传输的最大数据量，也称最大数据传输率。计算时应使用：

$$\text{总线带宽} = \text{总线宽度} \times \text{总线工作频率}$$

如果总线宽度以 bit 为单位，则要换算为 Byte：

$$\text{总线带宽} = \frac{\text{总线宽度}(\text{bit})}{8} \times \text{总线工作频率}$$

考点追踪：总线带宽的分析与计算（2009、2013、2014、2018—2020、2024、2025）

例如，若总线时钟频率为 11MHz，每个时钟周期可传送 2 次数据，则总线工作频率为：

$$11\text{MHz} \times 2 = 22\text{MHz}$$

若总线宽度为 16 位，即 2B，则总线带宽为：

$$2\text{B} \times 22 \times 10^6 = 44\text{MB/s}$$

这里要注意，计算带宽时不能只看两个设备之间一次通信的“有效速率”，而应依据总线的物理参数和传输机制，如总线宽度、时钟频率、每周期传输次数等。

6. 总线复用

总线复用指同一组信号线在不同时间段传输不同类型的信息。例如，地址线 and 数据线复用时，前一阶段传输地址，后一阶段传输数据。

总线复用的优点是减少引脚数量、降低硬件成本；缺点是不同信息要分时传输，控制更复杂，传输效率可能下降。

7. 总线寻址能力

总线寻址能力由地址线位数决定。若地址线有 n 位，且按字节编址，则最大可寻址空间为：

$$2^n\text{B}$$

例如，16 位地址线可表示 $2^{16} = 65536$ 个不同地址，若每个地址对应 1B，则最大寻址空间为 64KB。

本小节最重要的三个指标是：**总线宽度、总线工作频率、总线带宽**。带宽计算题往往会把“时钟频率”和“工作频率”分开给出，读题时必须先确认每个时钟周期传输几次数据。

6.1.4 总线的结构

总线结构是指计算机系统中各功能部件之间通过总线连接的组织方式。总线结构直接影响系统整体性能、扩展能力和瓶颈位置。

从发展过程看，总线结构经历了从**简单共享**到**分层并行**，再到更高集成和专用高速互连的演变。其核心目标始终是：提高带宽、降低延迟、减少通信冲突。

1. 早期共享总线结构：单总线结构

早期计算机常采用单总线结构：CPU、主存储器和各类 I/O 设备都挂接在同一条系统总线上。

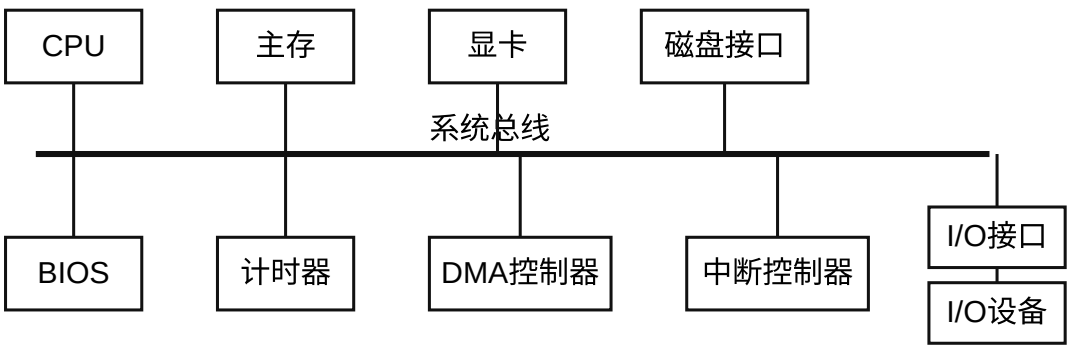


图6.1 单总线结构示意图

单总线结构的优点是实现简单、成本低、扩展直观；缺点是所有设备共享同一条总线，任意两个部件通信都可能占用总线，容易产生频繁的传输冲突。

在单总线结构中，CPU 访问主存、I/O 设备传输数据、DMA 控制器访存等操作都要竞争系统总线，因此系统吞吐率受总线带宽限制明显。当高速外设不断增加时，单总线很难满足数据吞吐需求。

2. 双总线结构

为了缓解 CPU 与主存之间的通信压力，可以引入双总线结构，即在系统中增设一条专用的存储总线，使 CPU 与主存之间的数据交换独立于 I/O 通路。

这种结构的改进点在于：CPU 与主存的通信不必总是和 I/O 操作竞争同一条总线，访存效率有所提高。

但它仍然存在明显问题：I/O 设备若要与主存交换数据，通常仍要通过 CPU 中转，CPU 可能成为新的性能瓶颈。

3. 三总线结构

进一步改进的三总线结构增加了 DMA 总线，允许高速 I/O 设备通过 DMA 控制器直接与主存通信，无须 CPU 逐字节干预。

这种结构能显著提高 I/O 吞吐能力，尤其适合磁盘、网络等大批量数据传输场景。但传统总线固有的共享和广播特性仍然会限制系统并发性和可扩展性。

4. 传统分层总线：南北桥结构

为突破早期共享总线的性能瓶颈，主流 PC 系统逐步转向分层次的多总线结构。传统代表是南北桥结构。

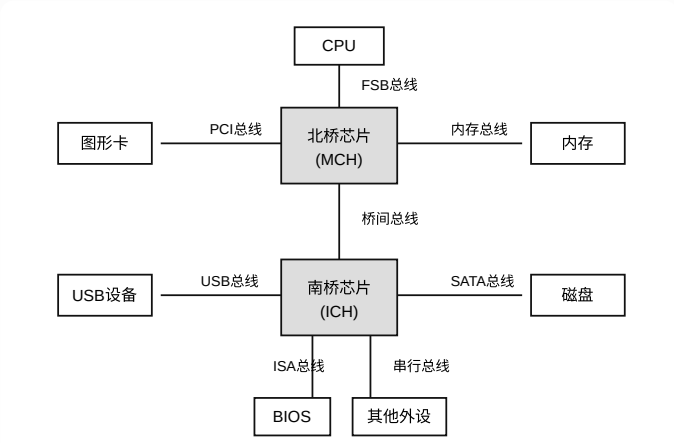


图6.2 南北桥结构示意图

南北桥结构将系统划分为两个主要功能区域：

部件	英文名称	主要连接对象	功能定位
北桥芯片	Memory Controller Hub, MCH	CPU、主存、显卡	高速枢纽，负责高带宽数据传输
南桥芯片	I/O Controller Hub, ICH	USB、SATA、以太网、低速外设、扩展槽等	I/O 控制器集线器，负责管理低速或中速 I/O 接口

在这种结构中，CPU 与北桥之间的互连通道称为前端总线，即 FSB，也称系统总线。CPU 通过 FSB 连接北桥，再经由北桥分别与主存和显卡通信。

各类 I/O 设备通过各自设备控制器连接到南桥，北桥与南桥之间通过专用桥间总线互连，最终形成从 I/O 到 CPU、主存的完整数据通路。

南北桥结构的本质是把高速访存与显示通路、低速 I/O 通路分层组织，减少所有设备共用单一总线带来的冲突。不过，跨层通信仍要经过桥接芯片，桥间带宽和桥接延迟也可能成为新的瓶颈。

6.2 总线事务和定时

本节讨论总线一次传输到底怎样发生，以及不同设备如何在时间上配合。总线不是“有线就能传”，而是要解决三个问题：**谁使用总线、一次怎样传、双方什么时候发和收。**

从考试角度看，本节重点是区分几组概念：非突发传输与突发传输、串行传输与并行传输、同步定时与异步定时。它们经常以选择题形式出现，也可能和总线带宽计算结合。

6.2.1 总线事务

总线事务是指主设备和从设备之间通过总线完成一次完整信息交换的过程。一次总线事务通常包含地址、命令、数据和响应等环节，其中主设备负责发起事务，从设备根据地址和控制信号作出响应。

为了理解总线事务，可以把它拆成“先说明访问谁，再说明做什么，最后传输数据”。例如 CPU 读主存时，大致过程是：CPU 给出主存地址和读命令，主存准备数据，数据稳定后送到数据总线，CPU 接收数据并结束本次事务。

1. 非突发传输

考点追踪：非突发传输的时间分析（2023）

非突发传输方式下，每次只传输一个数据单元，通常是一个总线宽度的数据。每次传输都要独立经历完整流程：发送地址、等待从设备准备数据、传输数据。

这种方式可以概括为：**一次地址，一次数据**。如果连续访问多个相邻地址，也必须重复发送地址，导致地址开销大、总线利用率较低。

例如连续读 4 个相邻数据时，非突发传输的逻辑是：

发送地址 A	-> 传输数据 D[A]
发送地址 A+1	-> 传输数据 D[A+1]
发送地址 A+2	-> 传输数据 D[A+2]
发送地址 A+3	-> 传输数据 D[A+3]

因此，非突发传输适合零散访问，但不适合连续块数据传输。

2. 突发传输

考点追踪：突发传输的特点与时间分析（2012—2014）

突发传输方式用于高速传输连续成块的数据。事务开始时，主设备只发送数据块的首地址；随后在不释放总线的前提下，连续传送多个数据单元。后续地址通常由硬件自动递增生成，例如首地址、首地址加 1、首地址加 2 等。

这种方式可以概括为：**一次地址，多次数据**。

```
发送首地址 A -> 连续传输 D[A], D[A+1], D[A+2], D[A+3], ...
```

突发传输的核心优势是减少重复发送地址的开销，显著提高有效带宽。它广泛用于高速存储器访问场景，例如 SDRAM 行读取、Cache 块填充等。

非突发传输和突发传输的区别可以这样记：

比较项	非突发传输	突发传输
地址发送	每传一个数据都要重新发送地址	只发送首地址，后续地址自动递增
数据传输	一次事务通常只传一个数据单元	一次事务连续传多个数据单元
总线占用	每次事务较短，但重复开销多	一次占用较长，但有效带宽高
适用场景	零散访问	连续块访问、Cache 块填充、高速存储器访问

3. 串行传输与并行传输

数据在线上的物理传输可采用串行或并行两种方式。

串行传输方式是数据按比特位依次顺序传输，通常只使用一条双向线路或两条单向线路。它的优点是引脚少、布线简单、抗干扰能力强，适合长距离通信，如 USB、PCIe、SATA 等。

在串行传输中，还可按时序协调方式分为同步串行通信和异步串行通信。

同步串行通信由发送方时钟直接控制接收方时钟，实现位同步。收发双方时钟严格一致，通常只在数据块首尾添加开始和结束标记。它的传输效率较高，但硬件实现复杂、成本较高。

考点追踪：异步串行通信方式的特点（2016）

异步串行通信中，收发双方使用独立时钟，不要求严格同步。每个字符独立传输，通常用起始位标志开始，用停止位标志结束。线路空闲时保持逻辑 1；发送字符时先发送逻辑 0 作为起始位，接收方检测到该低电平后开始接收数据。数据位一般从低位到高位逐位发送，数据位之后可选择发送一位奇偶校验位，最后发送停止位。

异步串行通信的典型格式可抽象为：

```
空闲状态(1) -> 起始位(0) -> 数据位 -> 可选校验位 -> 停止位(1)
```

并行传输方式是利用多条数据线同时传输多个比特位，例如 32 位、64 位总线可在一个传输周期内并行传输多个比特。理论上并行传输一周期即可完成一个字的传输，短距离内延迟低、吞吐高。

但并行传输并不总是比串行传输更快。频率升高后，多条并行线之间的信号串扰、时序偏移和布线复杂度会成为瓶颈，导致并行总线工作频率难以继续提高。现代高速接口常通过提高串行频率、多通道并行化和全双工传输来获得更高总带宽。

注意：并行传输并不总是比串行传输更快。受电气特性影响，并行总线的工作频率难以持续提高；串行传输可通过提高频率等方式获得更高总带宽，因此现代高速接口多采用串行化设计。

6.2.2 总线定时

总线定时是指总线上主设备与从设备在交换数据时，用于协调双方操作时序的控制协议。其实质是一种时序规则，用来规定发送方向时发、接收方向时收、等待条件如何判断、传输何时结束。

考点追踪：各种总线定时方式的特点（2015、2021）

常见总线定时方式包括同步定时、异步定时、半同步定时和分离式定时。

1. 同步定时方式

同步定时方式采用系统统一的时钟信号协调发送方和接收方的传送关系。时钟产生相等的时间间隔，每个时间间隔构成一个总线周期，每次数据传送在一个总线周期内完成。由于使用统一时钟，所有操作必须严格在固定周期内完成，总线周期连续运行。

优点：传送速度快，具有较高的传输速率，总线控制逻辑简单。

缺点：主、从设备之间属于强制性同步，所有操作严格受时钟节拍约束；缺乏应答或握手机制，无法根据从设备实际状态动态调整节拍，可靠性相对较低。

适用场景：适用于总线长度较短，并且连接各部件存取时间比较接近的系统。

2. 异步定时方式

异步定时方式不依赖统一时钟信号，而是通过主、从设备之间的握手信号实现定时控制。主设备发出“请求”信号；从设备准备就绪后，发出“回答”信号。

优点：总线周期长度可变，能可靠连接工作速度差异较大的设备，自适应性强。

缺点：控制过程复杂，需要额外握手信号和状态判断，每次信号交互都有延迟，因此整体传输速率较低。

根据“请求”和“回答”信号的撤销是否互锁，异步定时可分为三类：不互锁、半互锁和全互锁。

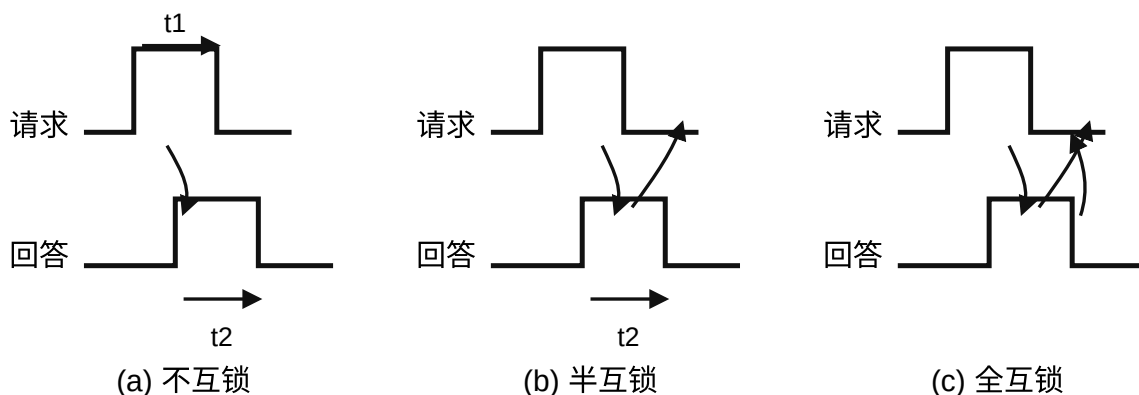


图6.3 请求和回答信号的互锁

类型	请求撤销条件	回答撤销条件	特点
不互锁方式	主设备发出请求后，在预设时延后自行撤销请求	从设备收到请求后立即回答，并在预设时延后自行撤销回答	双方无互锁关系，速度快但可靠性最低
半互锁方式	主设备必须收到回答后才能撤销请求	从设备发出回答后，无须确认请求是否撤销，在预设时延后自动撤销回答	请求撤销受回答约束，可靠性高于不互锁
全互锁方式	主设备必须收到回答后才能撤销请求	从设备必须确认请求已撤销后，才撤销回答	双方完全互锁，可靠性最高但速度较慢

适用场景：适用于连接速度差异大、对可靠性要求较高而对速率要求不苛刻的系统。

3. 半同步定时方式

半同步定时方式结合了同步与异步的优点。地址、命令和数据的发送严格参照系统时钟前沿，接收方通常在时钟后沿进行识别；同时增加一条 `wait` 信号线，允许慢速从设备反馈准备状态。

主设备在时钟上升沿检测 `wait` 信号状态。若 `wait` 无效，表示数据未就绪，主设备插入等待周期；直到 `wait` 有效，才从数据线读取数据。

优点：在统一时钟下工作，控制比异步方式简单，可靠性较高。

缺点：系统时钟频率受最慢设备限制，整体速度不高。

适用场景：适用于包含多种速度差异较大设备、但性能要求不高的简单系统。

同步、异步、半同步三种方式都属于“独占式事务模型”：从主设备发起请求到传送结束，总线全程被该事务占用。问题在于，从设备准备数据期间，总线也被占用但处于空闲等待状态，造成资源浪费。

4. 分离式定时方式

分离式定时方式把一个总线事务拆分为两个独立子阶段：请求阶段和应答阶段。两个阶段之间总线可以释放出来，允许其他主设备使用总线。

这种方式的思想是：主设备先发出访问请求，然后释放总线；从设备准备好数据后，再重新获得总线并返回结果。这样可以避免在等待从设备响应时长期占用总线，提高总线利用率。

它适用于总线事务响应时间较长、系统中存在多个主设备、希望提高并发度和总线利用率的场景。

本节的核心比较可以总结为：

定时方式	是否统一时钟	是否握手	周期长度	速度	可靠性/适应性	典型适用
同步定时	是	否	固定	快	较低	短总线、设备速度接近
异步定时	否	是	可变	较慢	高	速度差异大的设备
半同步定时	是	借助 <code>wait</code>	可插入等待周期	中等或偏低	较高	设备速度不一致的简单系统

定时方式	是否统一时钟	是否握手	周期长度	速度	可靠性/适用性	典型适用
分离式定时	不强调统一时钟	可按阶段控制	拆分为请求和应答阶段	利用率高	适合并发	多主设备、长响应事务

复习时可以用一句话抓住区别：**同步靠统一时钟，异步靠握手，半同步靠时钟加等待，分离式靠拆分事务释放总线。**

第七章 输入/输出系统

7.1 I/O系统基本概念

7.1.1 输入/输出系统

输入/输出系统关注的是**主机与外部世界之间的信息交换**。从主机视角看，输入是外部设备把信息送入主机，输出是主机把信息送到外部设备。I/O系统的核心任务不是单纯“传数据”，而是让速度、格式、电平、控制方式都不同的设备能够被CPU和主存统一管理。

本章需要把握两个主线：

- 1. **I/O 接口解决“能不能接、怎么接”的问题**：它负责设备选择、数据缓冲、状态反馈、控制命令传递、信号格式转换等。
- 2. **I/O 方式解决“CPU 怎样参与传输”的问题**：程序查询方式CPU参与最多，程序中断方式减少CPU等待，DMA方式进一步让大块数据传送绕开CPU的逐字干预。

I/O系统涉及的基本概念如下。

概念	含义	学习重点
外部设备	包括输入设备、输出设备，以及需要通过 I/O 接口访问的外部存储设备	设备本身不是 CPU 能直接统一控制的对象
接口	位于外设与主机之间的逻辑部件	负责速度匹配、电平转换、格式适配和控制协调
输入设备	向计算机输入命令、文本或数据的装置	如键盘、鼠标
输出设备	将计算机处理结果输出给用户的装置	如显示器、打印机
外存设备	主存和 CPU 缓存以外的存储器	如磁盘、光盘、固态硬盘等

通常，I/O系统由**I/O 硬件**和**I/O 软件**两部分组成。

- 1. **I/O 软件**：主要包括设备驱动程序、I/O 管理程序以及用户应用程序等，负责把用户或操作系统层面的 I/O 请求转化为硬件能执行的控制过程。
- 2. **I/O 硬件**：包括外部设备、设备控制器、I/O 接口以及 I/O 总线等。其中设备控制器偏向控制外设具体操作，I/O 接口偏向完成外设与主机总线之间的连接和适配。

容易混淆的是：外设不能直接和CPU“随便通信”。CPU通常通过总线访问I/O接口，而接口再去控制外设。接口相当于主机和外设之间的“翻译员 + 缓冲站 + 状态登记员”。

7.1.2 外部设备

常见外部设备主要包括键盘、鼠标、显示器、打印机、磁盘存储器和光盘存储器等。复习时不应把重点放在罗列设备名称，而应理解各种设备在I/O系统中的角色：输入设备产生数据，输出设备消费数据，外存设备既可能读也可能写，并且通常需要更复杂的控制过程。

1. 输入设备

键盘是最常见的输入设备，用于输入字符、数字和控制命令。鼠标属于定位输入设备，它把用户的移动、点击等操作映射为屏幕上的光标位置和交互动作。

2. 输出设备

显示器是最常见的输出设备，按显示器件可分为阴极射线管显示器（CRT）、液晶显示器（LCD）和发光二极管显示器（LED）等。显示器通常以点阵方式工作，核心性能参数包括：

参数	含义	考点理解
屏幕尺寸	通常用对角线长度表示，单位为英寸	主要是物理尺寸，不直接等于显示清晰度
分辨率	屏幕可显示的像素总数，常表示为“宽度 × 高度”	决定一帧图像包含多少像素
色彩深度	每个像素用多少位二进制表示	色彩越丰富，每帧图像所需显存越大
刷新机制	像素发光持续时间短，需要周期性重绘画面	保持画面稳定
刷新频率	每秒刷新屏幕的次数	一般支持 60—120Hz，频率过低会产生闪烁感

考点追踪：显示刷新带宽的计算（2010）。

显示存储器（VRAM，也称刷新存储器）用于存储一帧完整的图像数据。计算时要注意单位：色彩深度通常以“位”为单位，而存储容量常以“字节”为单位。

$$\begin{aligned} \text{VRAM容量 (bit)} &= \text{分辨率} \times \text{色彩深度 (位数)} \\ \text{VRAM带宽 (bit/s)} &= \text{分辨率} \times \text{色彩深度 (位数)} \times \text{帧频} \end{aligned}$$

若要求字节数，还需要除以 8。做题时常见陷阱是把 bit 和 Byte 混用，或漏乘刷新频率。

打印机用于把计算机输出结果打印到纸张等介质上。按工作原理可分为以下几类。

类型	基本原理	特点
针式打印机	通过打印针撞击色带在纸上形成字符	可多层复写、耗材便宜，但噪声大、分辨率低、速度慢
喷墨打印机	通过精确喷射微小墨滴形成图像	成本较低，图像效果较好，家用和办公常见
激光打印机	利用激光扫描和静电成像技术形成图像，再经显影、转印、定影输出	速度快、质量高、处理能力强，办公环境常用

3. 外部存储器（辅助存储器）

外部存储器属于辅助存储器，通常用于长期保存程序和数据。它与主存相比容量大、单位成本低，但访问速度慢，需要通过 I/O 接口或设备控制器间接访问。

磁表面存储器利用磁性材料记录信息，典型代表包括硬盘、软盘、磁带和磁鼓等。理解外存时要特别关注“它不是主存的一部分”：CPU 不能像访问寄存器或主存单元那样直接访问外存中的每个字节，而是要通过 I/O 控制过程完成读写。

7.2 I/O接口

7.2.1 I/O接口的功能

I/O 接口也称 I/O 控制器，是连接主机和外设的关键部件。它的存在本质上是因为 CPU、主存和外设在速度、控制方式、信号格式、电平标准等方面并不一致。

考点追踪：I/O 接口的定义与特性（2021）。

I/O 接口的主要功能如下。

功能	作用	典型理解
地址译码与设备选择	CPU 发出 I/O 操作请求时，接口提供或识别目标外设的地址码，并选中对应设备	“找对设备”
通信联络与时序协调	通过握手信号、状态轮询等机制协调主机和外设节奏	“快慢设备能配合”
数据缓冲	使用数据缓冲寄存器暂存传输数据	“速度不匹配时先临时存放”
信号格式转换	完成电平转换、串并转换、模数或数模转换等	“主机和外设信号能互相理解”
控制命令与状态传递	CPU 通过控制寄存器给外设发命令，外设通过状态寄存器反馈状态	“CPU 能下命令，也能知道外设忙不忙、是否就绪”

这里最容易出选择题的是“接口到底传什么”。接口不仅传数据，还传控制信息和状态信息；也可能传中断请求、总线仲裁、握手信号等与 I/O 控制有关的信息。

7.2.2 I/O接口的基本结构

考点追踪：I/O 端口与 CPU 交换的内容（2015）。

一个通用 I/O 接口通常位于主机侧系统总线和设备侧接口电缆之间。其内部核心是若干寄存器和控制逻辑，外部则分别连接 CPU、内存和外设。

I/O 接口内部常见的三类寄存器如下。

寄存器	保存内容	访问特点
数据缓冲寄存器	暂存 CPU 与外设之间传送的数据	读写数据时使用
状态寄存器	记录接口和外设当前状态，如就绪、忙、错误等	通常由 CPU 读取
控制寄存器	保存 CPU 发给外设的控制命令	通常由 CPU 写入

在某些设计中，可复用同一端口地址：读操作访问状态信息，写操作访问控制信息。这样做的前提是 CPU 读写方向不同，接口可以根据读/写控制信号区分访问目标。

I/O 接口与系统总线交换的信息可概括为：

数据总线：数据、状态信息、控制信息、中断类型号

地址总线：要访问的 I/O 端口地址

控制总线：读/写控制信号、中断请求/响应信号、总线仲裁信号、握手信号等

考点追踪：I/O 接口的数据线上传输的内容（2012）。

其中，数据线不只传输“数据”。因为状态字、控制字、中断类型号本质上也需要以二进制编码形式在总线上传递，所以它们也可能经过数据总线传输。地址线负责指出访问哪个端口，控制线负责说明访问方向、时序和控制行为。

I/O 控制逻辑是接口内部的“执行中心”，主要完成三类工作：

1. 对控制寄存器中的命令字进行译码，并把形成的控制信号送往外设。
2. 将数据缓冲寄存器中的内容送给外设，或把外设输入的数据写入数据缓冲寄存器。
3. 实时采集外设状态，并更新状态寄存器，供 CPU 查询或供中断逻辑使用。

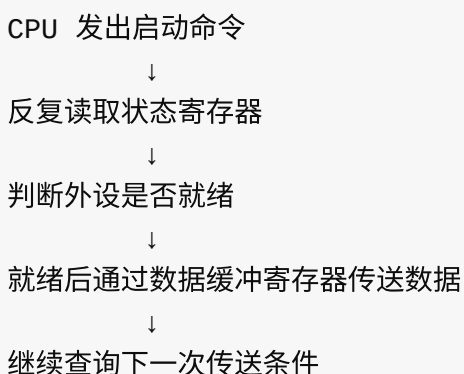
从考试角度看，I/O 接口的基本结构可抓住一条线：**CPU 通过端口访问寄存器，接口通过控制逻辑管理外设**。寄存器解决“存什么”，控制逻辑解决“怎么执行”，总线信号解决“如何和 CPU 通信”。

7.3 I/O方式

7.3.1 程序查询方式

程序查询方式的核心思想是：**CPU 主动、反复查询 I/O 接口状态寄存器**，当外设就绪后，再执行数据传送。也就是说，外设不会主动打断 CPU，而是 CPU 不断“问外设好了没有”。

程序查询方式的一般过程可以概括为：



这种方式的优点是**硬件控制简单**，接口电路和控制逻辑不复杂，适合低速、简单外设或对实时性要求不高的场景。它的缺点也很明显：CPU 需要花大量时间进行查询与等待，且同一时间段内通常只能和一台外设通信，导致 CPU 与外设串行工作，效率较低。

一个典型效率计算思路是：先算 CPU 每秒为 I/O 查询消耗的时钟周期数，再除以 CPU 每秒总时钟周期数。若某题中每秒有 5×10^5 次 I/O 相关操作，每次需要 10 次查询，每次查询平均消耗 4 个时钟周期，CPU 主频为 500MHz，则 I/O 查询占用比例为：

$$\frac{5 \times 10^5 \times 10 \times 4}{5 \times 10^8} = 4\%$$

这个例子说明，程序查询方式不是一定“不能用”，而是要看查询频率和单次查询开销。如果查询很频繁，CPU 时间会被大量消耗；如果外设很低速、查询间隔较长，则开销可能可以接受。

7.3.2 程序中断方式

程序中断方式的基本思想是：CPU 不再一直等待外设，而是先继续执行当前程序；当外设完成准备或发生需要处理的事件时，外设通过接口向 CPU 发出中断请求，CPU 在适当时机暂停当前程序，转去执行中断服务程序，处理完后返回原程序断点继续执行。

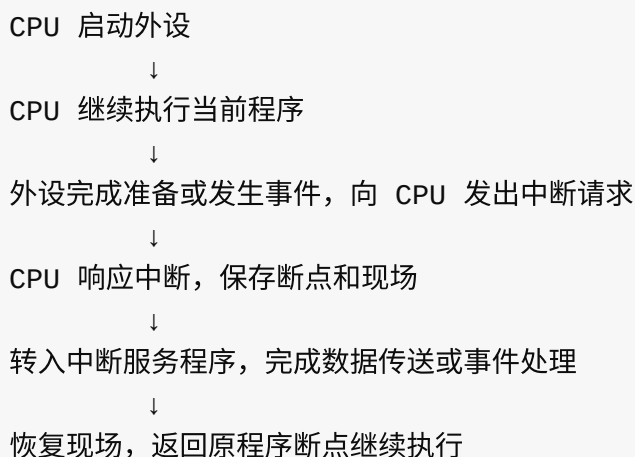
它解决了程序查询方式中 CPU 长时间“忙等”的问题，使 **CPU 与外设可以并行工作**。但注意：中断方式只是减少等待，并不是传送过程完全没有 CPU 开销，因为中断响应、现场保存、中断服务和恢复现场仍需要 CPU 参与。

考点追踪：中断方式的特点（2022、2023）。

程序中断技术最初主要用于主机与 I/O 设备之间的异步通信。随着计算机系统发展，中断机制还承担了更多功能：

1. 实现 CPU 与 I/O 设备的并行工作。
2. 处理硬件故障和软件异常。
3. 支持人机交互，如键盘输入、鼠标点击。
4. 支持多道程序和分时操作系统中的任务切换。
5. 满足实时系统对快速响应的要求。
6. 实现用户程序向操作系统的切换，如软中断或系统调用。
7. 在多处理器系统中协调处理器之间的通信与任务迁移。

程序中断方式的基本工作过程可以理解为下面这条主线：



例 7.2 的典型分析：中断是否适合高速外设

已知 CPU 主频为 500MHz，某外设传输速率为 40MB/s，I/O 接口只有一个 32 位数据缓冲寄存器，每次中断响应与中断处理共需 400 个时钟周期。

中断响应与处理时间为：

$$\frac{400}{500 \times 10^6} = 0.8\mu s$$

接口缓冲寄存器为 32 位，即 4B。外设以 40MB/s 速度填满 4B 缓冲区所需时间约为：

$$\frac{4B}{40MB/s} = 0.1\mu s$$

由于 $0.1\mu s < 0.8\mu s$ ，外设产生数据的速度远快于 CPU 完成一次中断服务的速度。如果每填满 32 位就触发一次中断，前一次中断尚未处理完，下一批数据就可能到达，导致缓冲区被覆盖而丢失数据。因此，该高速外设不适合直接采用“单字中断”方式；若仍采用中断，应扩大缓冲区或采用 DMA 等更适合大批量高速传送的方式。

考点追踪：程序中断的效率分析及相关计算（2009、2014、2016、2018、2019）。

程序中断的工作流程可以按“请求、判优、响应、服务、返回”来记。

1. 中断请求

中断请求是指能够向 CPU 发出中断请求的设备或事件，请求 CPU 暂停当前执行流并转入相应处理程序。实际系统中可能同时存在多个中断源，而且中断到来的时间具有随机性。

为记录某个中断源是否正在请求中断，系统通常设置中断请求标记触发器。当其状态为 1 时，表示该中断源有请求。这些触发器可以集中组成中断请求标记寄存器，也可以分散在各个中断源中。

中断按是否可以被屏蔽，可分为：

类型	常见信号	特点	典型事件
可屏蔽中断	INTR	受中断允许标志或屏蔽字控制，CPU 可选择暂不响应	普通 I/O 中断
不可屏蔽中断	NMI	优先级高，不受中断允许标志限制	电源掉电、总线错误等严重硬件事件

考点追踪：可屏蔽中断和不可屏蔽中断的特点（2020）。

复习时要注意：可屏蔽中断一般用于普通外设请求；不可屏蔽中断通常表示不能延后的严重事件，所以优先级更高。

2. 中断响应判优

当多个中断源同时请求中断时，CPU 必须决定先响应哪一个，这就是中断判优。判优可以由硬件排队电路完成，也可以由中断查询程序完成。

一般原则如下：

1. 不可屏蔽中断优先于可屏蔽中断。
2. 在 I/O 传送类中断中，高速设备优先于低速设备。
3. 输入设备通常优先于输出设备。
4. 实时设备通常优先于普通设备。

这里要区分两个概念：**响应优先级**和**处理优先级**。响应优先级表示 CPU 在多个请求同时到来时先响应谁；处理优先级表示已经进入某个中断服务程序后，是否允许更高优先级中断打断当前中断服务程序。若不支持多重中断，响应优先级和处理优先级往往表现得一致；若支持中断屏蔽和中断嵌套，两者可能不同。

3. CPU 响应中断的条件

CPU 只有在满足一定条件时才会响应中断请求。一般需要同时满足：

1. 存在有效的中断请求。
2. CPU 处于开中断状态，即中断允许标志为 1。异常和不可屏蔽中断通常不受该条件限制。
3. 当前指令已经执行完毕，并且不存在更高优先级的任务需要处理。

考点追踪：CPU 响应中断的条件（2023）。

普通 I/O 中断的响应时机通常发生在某条指令执行结束之后。也就是说，CPU 不会在一条普通指令执行到一半时随意响应 I/O 中断。异常则不同，例如缺页、除零等异常可能和当前指令执行过程直接相关。

4. 中断响应过程

CPU 响应中断后，会由硬件自动完成一系列操作，这些操作常称为**中断隐指令**。它不是指令系统中可见的一条普通机器指令，而是硬件在中断响应阶段自动执行的一组动作。

中断隐指令主要完成三项工作：

1. **关中断**：将中断允许标志置为禁止状态，防止在保存断点和现场期间又被新的中断打断。
2. **保存断点**：把返回原程序所需的地址保存起来，通常保存程序计数器 *PC* 的内容；有些机器还会同时保存程序状态字 *PSW*。
3. **引出中断服务程序**：根据中断源识别结果，把对应中断服务程序的入口地址送入 *PC*，使 CPU 转去执行中断服务程序。

断点地址要结合事件类型理解：故障类异常如果在指令未完成时发生，处理后通常需要重新执行当前指令，断点常为当前指令地址；陷阱类异常如系统调用，在指令成功执行后触发，断点通常为下一条指令地址。

5. 中断向量

中断识别可以分为向量中断和非向量中断。非向量中断常用软件查询法；向量中断则为每个中断源分配一个唯一的 interrupt type，该 interrupt type 对应一个中断服务程序入口地址。

在向量中断中，所有中断向量集中存放在内存的特定区域，这个区域称为**中断向量表**。CPU 响应中断后，通常先取得中断源的中断类型号，再据此找到中断向量表中的相应表项，将该表项中的入口地址送入 *PC*，从而转入对应的中断服务程序。

考点追踪：中断向量表的数据结构（2023）。

容易混淆的是信号所在总线：中断请求和响应信号通常通过 I/O 总线的控制线传输；而中断类型号是在中断响应阶段由中断控制器经数据总线提供给 CPU，用于定位中断向量表中的表项。

6. 中断处理过程

一个支持中断嵌套的典型中断处理流程如下：

步骤	操作	主要完成者	作用
1	关中断	硬件	防止保存关键信息时被再次中断
2	保存断点	硬件	保存返回原程序所需地址
3	中断服务程序寻址	硬件	根据中断源找到服务程序入口
4	保存现场和中断屏蔽字	软件	保存通用寄存器等现场信息，便于后续恢复
5	开中断	软件	允许更高优先级中断打断当前服务程序，实现嵌套
6	执行中断服务程序	软件	完成数据传送、事件处理或故障处理
7	关中断	软件	防止恢复现场时被新中断破坏
8	恢复现场和中断屏蔽字	软件	恢复被中断程序的运行环境
9	开中断、中断返回	软件	返回断点，继续执行原程序

考点追踪：中断隐指令的功能（2012）。

考点追踪：单重中断的处理流程（2010）。

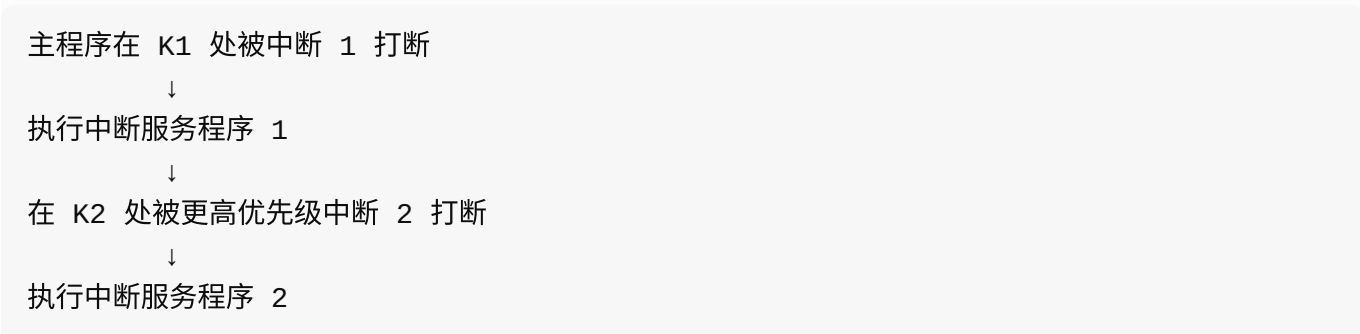
其中，第 1—3 步由中断隐指令通过硬件自动完成；第 4—9 步通常由中断服务程序通过软件完成。若系统只支持单重中断、不支持中断嵌套，则流程中的“开中断”和后面的“关中断”可以省略，因为服务程序执行期间不允许新的中断打断。

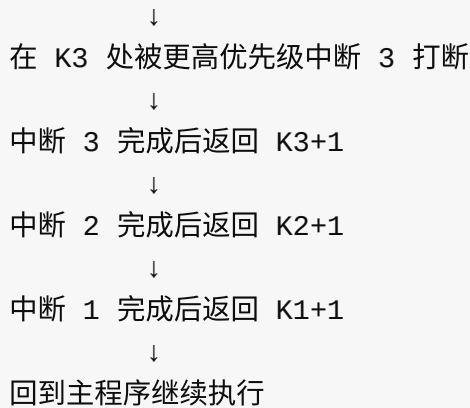
需要特别区分**断点与现场**：断点是返回原程序所需的地址信息，通常由硬件保存；现场是可能被中断服务程序修改、恢复后才能让原程序正确继续执行的寄存器内容，通常由软件显式保存。

7. 多重中断和中断屏蔽技术

如果 CPU 执行中断服务程序期间不响应新的更高优先级中断，称为单重中断；如果允许更高优先级中断打断正在执行的中断服务程序，称为多重中断，也称中断嵌套。

多重中断的执行轨迹可以理解为“后进先出”的栈式恢复：





考点追踪：多重中断的中断屏蔽字相关性质（2017、2020、2023）。

中断处理优先级由中断屏蔽字控制。每个中断源通常对应屏蔽字中的一位，若该位为 1，表示屏蔽对应中断源；若该位为 0，表示允许对应中断源进入判优电路。通过在不同中断服务程序中装入不同屏蔽字，可以动态调整“谁能打断谁”。

要抓住两点：

1. 中断屏蔽字一般在进入中断服务程序时由软件装入，用于控制后续中断嵌套行为。
2. 中断屏蔽字在 CPU 执行中断服务程序期间生效；主程序运行阶段是否响应中断，通常由主程序设置的中断允许状态决定。

如果不使用屏蔽技术，则处理优先级通常与响应优先级一致；采用屏蔽技术后，处理优先级可以和响应优先级不同。

例 7.3 的典型分析：由处理优先级构造屏蔽字

设中断源为 A、B、C、D，硬件响应优先级为 $A > B > C > D$ ，要求实际中断处理优先级调整为 $A > D > C > B$ 。设计屏蔽字时，可按“本中断及低于本中断处理优先级的中断都屏蔽，只允许更高处理优先级中断打断”的思路构造。

中断源	A	B	C	D
A	1	1	1	1
B	0	1	0	0
C	0	1	1	0
D	0	1	1	1

按这个表理解：A 的处理优先级最高，因此 A 服务期间屏蔽所有中断；D 的处理优先级仅低于 A，因此 D 服务期间只允许 A 打断；C 服务期间允许 A 和 D 打断；B 的处理优先级最低，因此 B 服务期间只屏蔽自身，允许 A、C、D 打断。这里表中的 1 表示屏蔽，0 表示不屏蔽。

例 7.3 的执行轨迹还可以继续按“先看响应优先级，再看屏蔽字是否允许嵌套”的顺序分析。若 CPU 正在执行用户程序时，A、C、D 同时发出中断请求，硬件响应优先级为 $A > B > C > D$ ，所以 CPU 先响应 A。进入 A 的服务程序后装入屏蔽字 1111，所有中断均被屏蔽，A 独占 CPU 直到服务完成。

A 返回用户程序后，C 和 D 的请求仍待处理。按响应优先级，CPU 先响应 C；进入 C 的服务程序后装入屏蔽字 0110，表示允许 D 打断 C，因此 CPU 在 C 的服务过程中响应 D。D 的处理过程中没有出现更高处理优先级的请求，所以 D 完成后返回被打断的 C 继续执行。C 执行期间 B 发出请求，但 C 的屏蔽字为 0110，B 被屏蔽，所以此时不响应 B。C 完成后返回用户程序，最后 CPU 再响应 B。可把这个过程记成下面的轨迹：

用户程序

↓ A、C、D 同时请求，按响应优先级先响应 A

A 服务程序：屏蔽字 1111，所有中断均被屏蔽

↓ A 完成并返回

用户程序

↓ C、D 未处理，按响应优先级先响应 C

C 服务程序：屏蔽字 0110，允许 D 打断 C，但屏蔽 B、C

↓ D 请求可响应

D 服务程序：完成后返回 C

↓ C 继续执行；B 请求被 C 的屏蔽字屏蔽

C 服务程序完成并返回用户程序

↓ 最后响应 B

B 服务程序完成并返回用户程序

从宏观角度看，程序中断方式克服了程序查询方式中 CPU 长时间忙等的问题，明显提高了 CPU 利用率；但从微观角度看，CPU 在处理中断时仍要暂停原程序，完成中断响应、现场保存、中断服务、现场恢复等工作。若高速外设频繁、成批地与主存交换数据，若每个字或每小块都触发中断，CPU 会被大量中断开销拖住，因此需要 DMA 方式。

7.3.3 DMA方式

DMA 方式是一种**主要由硬件控制的数据传送方式**。它的核心思想是：在外设和内存之间建立直接的数据通路，使成批数据传送不再逐字经过 CPU。这样，在数据准备阶段，CPU 可以与外设并行工作；在数据传送阶段，DMA 控制器直接接管总线完成主存与外设之间的数据交换；传送结束后，再通过中断通知 CPU 做收尾处理。

考点追踪：DMA 方式的使用场景（2025）。

DMA 特别适合磁盘、显卡、声卡、网卡等高速设备的大批量数据传输。这类设备的数据率较高，若采用程序查询或逐字中断方式，CPU 会被大量等待或中断服务占用。DMA 的优势就在于把“每个数据字都要 CPU 参与”的传送过程，改为“CPU 只负责初始化和结束处理，中间大段传送由硬件完成”。

考点追踪：DMA 方式中的数据传输通路（2024）。

1. DMA 方式的特点

DMA 的关键特点可以概括为：**直接通路、硬件计数、成批传送、CPU 少参与**。

特点	含义	考点理解
外设与主存直接交换数据	数据传送不经过 CPU 内部寄存器	减少 CPU 逐字搬运数据的负担
主存既能被 CPU 访问，也能被外设通过 DMA 访问	DMA 打破了主存只由 CPU 固定访问的关系	会引出 CPU 与 DMA 争用主存的问题
地址生成和字数计数由硬件完成	DMA 控制器自动修改主存地址、递减计数器	不需要 CPU 每传一个字都参与
主存中常设置专用缓冲区	用于成批数据交换	便于高速设备与主存之间批量传输
支持高速数据传输	CPU 与外设可在较大范围内并行工作	显著提高系统效率
传送开始和结束仍需 CPU 参与	开始前由 CPU 初始化，结束后通过中断通知 CPU	DMA 不是完全不需要 CPU

需要注意，DMA 与程序中断方式并不是完全对立的关系。DMA 传输过程中通常不需要 CPU 执行中断服务程序，但**DMA 传送结束或出现异常时，仍常用中断通知 CPU。**

2. DMA 控制器的组成

在 DMA 方式中，对数据传送过程进行硬件控制的部件称为 **DMA 控制器**，也可称 DMA 接口。当外设需要进行 DMA 传送时，DMA 控制器向 CPU 提出总线请求；CPU 响应后暂时让出系统总线，由 DMA 控制器接管总线完成数据传送。

DMA 控制器的主要功能可以按工作顺序理解：

1. 接收外设发出的 DMA 请求，并向 CPU 发出总线请求信号。
2. 在 CPU 发出总线响应信号后，接管总线控制权，进入 DMA 操作周期。
3. 根据 CPU 预先设置的参数，确定主存起始地址、传送方向和传送长度。
4. 发出读/写控制信号，在主存与外设之间完成数据传送。
5. 数据块传送结束后，向 CPU 报告传送完成或异常情况。

DMA 控制器内部常见部件如下。

部件	作用	记忆方式
主存地址计数器	保存当前要访问的主存地址，每传送一个字自动修改地址	“下一个主存位置在哪”
传送长度计数器	保存剩余传送字数，每传送一个字递减，减到 0 表示结束	“还剩多少字没传”
数据缓冲寄存器	暂存每次传送的数据	“中转站”
设备地址寄存器	保存 I/O 设备地址或设备选择信息	“目标设备是谁”
控制/状态逻辑	设定传送方向，协调 DMA 请求、CPU 响应和读写控制	“控制中心”
中断机构	数据块传送完成或异常时向 CPU 发中断请求	“收尾通知”

DMA 传送时，DMA 控制器接管系统总线并执行数据传送；传送结束后，再把总线控制权交还给 CPU。正因为 DMA 控制器需要独立控制系统总线，所以它必须具备一定的总线控制能力。

3. DMA 的传送方式

DMA 使外设可以直接和主存交换数据，但当 DMA 控制器与 CPU 同时访问主存时，可能发生总线或主存访问冲突。为了解决这个问题，DMA 与 CPU 通常采用以下三种方式共享主存。

方式	基本做法	优点	缺点	适用场景
停止 CPU 访存	DMA 请求到来后，CPU 暂停访问主存，直到 DMA 完成成组数据传送	控制逻辑简单，适合高速成组传送	DMA 期间 CPU 基本空闲，资源利用率低	高速 I/O 设备一次传送整块数据
周期挪用	DMA 只在需要传送一个字时占用一个或几个主存周期	兼顾 I/O 数据传送需求与 CPU 工作效率	每次挪用都要申请、释放总线，带来一定开销	高速 I/O、单字传送，常见考点
DMA 与 CPU 交替访存	把 CPU 工作周期划分为两个子周期，一个给 CPU 访存，一个给 DMA 访存	不需要申请和释放总线，传送速率高	硬件控制逻辑复杂	CPU 工作周期长于主存存取周期的系统

停止 CPU 访存方式中，DMA 控制器向 CPU 发出停止信号，CPU 放弃总线控制权并暂停访存，直到 DMA 完成整块数据传送后再恢复主存访问。它简单但粗暴，DMA 期间 CPU 基本不能有效工作。

周期挪用也称周期窃取。对于高速 I/O 设备，如果不及时访问主存就可能丢失数据，因此 DMA 访存请求具有较高优先级。若 CPU 当前没有访存，DMA 可直接使用总线；若 CPU 正在访存，则等待当前存储周期结束后暂时取得总线；若 I/O 与 CPU 同时请求访存，则 CPU 暂时让出一个存储周期。周期挪用的关键不是长时间停掉 CPU，而是“借用少量主存周期完成单字传送”。

考点追踪：周期挪用的特点及挪用次数分析（2012、2020、2022）。

DMA 与 CPU 交替访存方式中，CPU 周期被分成两个子周期：一个专供 CPU 访存，另一个专供 DMA 访存。这样两者按固定节拍轮流访问主存，不需要总线控制权的申请和释放，数据传送速率较高。它适用于 CPU 工作周期长于主存存储周期的场景；代价是硬件控制逻辑更复杂。

考点追踪：DMA 方式的效率分析及相关计算（2011、2018）。

例 7.4 的典型分析：DMA 方式下 CPU 时间占比

已知计算机主频为 500MHz，某外设数据率为 40MB/s，采用 DMA 方式，每次 DMA 传送块大小为 1000B，DMA 预处理和后处理总共需要 500 个时钟周期。问 CPU 用于该外设 I/O 的时间占比。每秒需要进行的 DMA 批次数为：

$$\frac{40\text{MB/s}}{1000\text{B}} = 40000 \text{ 次/s}$$

CPU 每秒用于 DMA 预处理和后处理的时钟周期数为：

$$40000 \times 500 = 2 \times 10^7$$

CPU 每秒总时钟周期数为：

$$500\text{MHz} = 5 \times 10^8$$

所以 CPU 用于该外设 I/O 的时间占比为：

$$\frac{2 \times 10^7}{5 \times 10^8} = 4\%$$

这里容易误用题中的 $CPI = 4$ 。本题问的是 CPU 总时间占比，已直接给出主频和 DMA 处理所需时钟周期，因此按时钟周期比例计算即可； CPI 与这类 DMA 预处理/后处理周期占比没有直接关系。

4. DMA 的传送过程

考点追踪：DMA 方式的传送过程（2019）。

DMA 的数据传送过程通常分为三个阶段：预处理、数据传送和后处理。

阶段	主要工作	CPU 是否参与
预处理	CPU 初始化 DMA 控制器，设置主存起始地址、设备地址、传送方向、传送字数，并启动 I/O 设备	参与
数据传送	DMA 控制器获得总线控制权后，自动完成主存与外设之间的数据块传送，并自动修改地址和计数	通常不参与
后处理	数据块传送完成后，DMA 控制器向 CPU 发中断请求，CPU 检查传送是否正确，必要时处理错误	参与

预处理阶段可以理解为 CPU 给 DMA 控制器“填任务单”：主存起始地址写入主存地址寄存器，I/O 设备地址写入设备地址寄存器，传送字数写入字数计数器，同时设置读/写方向并启动外设。随后 CPU 继续执行原程序。

数据传送阶段由 DMA 控制器主导。I/O 设备准备好发送或接收数据后，向 DMA 控制器发出 DMA 请求；DMA 控制器再向 CPU 申请总线控制权。获得总线后，DMA 控制器把主存地址送上地址总线，控制数据在主存与 I/O 设备之间传送，并在每次传送后自动修改主存地址和字数计数器。若字数计数器未到结束条件，则继续传送；若数据块传送结束，则进入后处理。

后处理阶段，DMA 控制器向 CPU 发出中断请求。CPU 响应后执行相应中断服务程序，检查数据完整性、处理传输错误或完成必要的软件收尾工作。

考点追踪：DMA 传送结束时的处理（2025）。

可以把三种 I/O 方式放在一起比较：

I/O 方式	CPU 参与程度	是否有忙等	适合设备	核心缺点
程序查询方式	最高，CPU 不断查询状态并执行传送	有	简单低速设备	浪费 CPU 时间

I/O 方式	CPU 参与程度	是否有忙等	适合设备	核心缺点
程序中断方式	中等，外设就绪后中断 CPU，CPU 执行服务程序	基本无忙等	中低速、随机事件型设备	中断响应和服务仍有开销
DMA 方式	最低，CPU 主要负责初始化和结束处理	无逐字忙等	高速、成批数据传送设备	硬件复杂，需要协调总线/主存争用

本章的复习主线可以压缩成一句话：**I/O 接口解决主机和外设如何连接，程序查询、中断、DMA 则按 CPU 参与程度逐步降低，分别适合低速简单、异步中速和高速成批传送场景。**